SAP-C02 Domain 4 — Accelerate Workload Migration and Modernization

Average question, answer template and traps for tasks 4.1–4.4 · built from the 159 Domain 4 questions in your trainer · D1 share · D2 build · D3 fix · **D4 = move from on-premises (and modernize) with the least risk, downtime and rework**

The average Domain 4 question

> "A company runs **[VMs / physical servers / Oracle or SQL Server / NFS or SMB shares / Kubernetes / a monolith]** in its data center. It must move to AWS **[within N months / with minimal downtime / without code changes / without managing servers]**. Which approach has the **LEAST operational overhead** (or is **MOST cost-effective**)?" — 115 of 159 mention on-premises; 38 say LEAST overhead.

The average answer has 4 parts

- **Where (what is moving):** servers, a database, files, containers or an app being redesigned — each has its own tool.
- **What (the tool):** servers → **MGN** · database → **DMS** (+ **SCT** if engine changes) · files → **DataSync** · keep on-prem file access → **Storage Gateway** · discover/plan → **AWS Transform** (exam: ADS / Migration Hub).
- **How it survives:** land in a managed, multi-AZ target (Aurora/RDS, Fargate, EKS managed nodes, Beanstalk, EFS/FSx).
- **How it's connected and paid for:** existing DX/VPN; DX gateway + TGW for many VPCs; licence choices (Babelfish, Aurora PostgreSQL, Dedicated Hosts BYOL).

The wording tells you which answer wins

- **LEAST overhead** → managed target (Fargate, Aurora, Transfer Family, Amazon MQ, SES), not EC2.
- **Minimal downtime / no data loss** → continuous replication then cutover (DMS full load + **CDC**, MGN, DataSync incremental).
- **Can't change code / no source** → rehost or replatform: MGN, Beanstalk + JAR, same engine, Amazon MQ, DocumentDB, EKS for manifests.
- **Eliminate licences** → Aurora PostgreSQL (SCT / DMS Schema Conversion) or Babelfish; .NET → Linux containers (Transform for .NET).
- **Stays N years / unused / SaaS** → retain / retire / repurchase.
- **First workload, team new** → low-risk pilot (rehost a simple internal app).
- **Runs over 15 min** → Fargate / Batch / ECS, not Lambda.

The 5 migration checks

Check	Rule
What is moving?	Server → MGN · database → DMS (+SCT) · files → DataSync · tape/iSCSI/SMB kept on-prem → Storage Gateway · inventory → Transform / Discovery Agent.
How long can it be down?	Hours → backup/restore. Minutes → continuous replication + cutover (DMS CDC, MGN).
Can the code change?	No → rehost / replatform, same engine and protocol. Yes → refactor to serverless or containers.
Where does it land?	Managed, multi-AZ; keep the protocol: SMB → FSx Windows, NFS → EFS, iSCSI → Volume Gateway, MongoDB → DocumentDB, Kafka → MSK, AMQP → Amazon MQ.
How does data travel?	Existing DX/VPN if there's time; a new DX takes weeks — start now with VPN.

Reading routine

- **Last sentence first:** deciding words, how many answers (20 ask for TWO or THREE).
- Underline: **source** (VMware, physical, Oracle, Windows, Kubernetes) · **limits** (deadline, downtime, no code change, no agents, compliance) · **network** (DX speed, internet only, time to provision).
- Cross out: wrong tool for what's moving, self-managed EC2 targets, rewrites on short timelines, the facts below.

Wrong almost every time: self-managed EC2 target when a managed service exists (59 wrong vs 18 right) · Lambda for long-running or stateful legacy apps (43 wrong vs 16 right) · **MGN for the wrong job** (26 wrong vs 3 right — it moves servers, not dependency maps, database-only data or file shares) · Storage Gateway for a one-time bulk copy (18 wrong vs 4 right — use DataSync) · one-time dump/snapshot when "no data loss" · waiting for a new Direct Connect · ALB for UDP or SFTP · EFS for Windows SMB · DynamoDB for relational data with stored procedures · big-bang migration by a new team · moving apps tied to on-prem hardware or the mainframe.
Status: Application Discovery Service, Migration Hub, Refactor Spaces, App2Container, Porting Assistant closed to new customers Nov 7, 2025 → AWS Transform · SMS retired → MGN · Snowball existing customers only, support ends Dec 31, 2026 · App Runner closed → ECS Express Mode · KDA for SQL ended → Managed Service for Apache Flink.

4.1 Select workloads for migration

Q: Before migrating, the company needs an inventory / dependency map / business case / strategy per app / wave plan.
A: Discover → business case → one of the 7 Rs per app → waves by dependency.

Need	Answer
Processes + network connections	**Discovery Agent** (exam) / **Transform discovery** (now)
VM utilization, no agents	**Agentless collector** / RVTools → **Transform**
TCO / business case incl. licensing	**Migration Evaluator** (or Transform)
Is the organization ready?	**Migration Readiness Assessment** (CAF)
Unused / must stay / swap for SaaS	**Retire / retain / repurchase**

Need	Answer
Team new to AWS	**Pilot wave**; chatty apps + DB move **together**
Reuse Windows/SQL licences by core	**Dedicated Hosts + License Manager**
Drop Oracle licences	**Aurora PostgreSQL** via Schema Conversion + DMS CDC
.NET Framework / mainframe code	**AWS Transform for .NET / mainframe**

Checks: inventory or business case? · which R per app? · do dependent apps move together?

4.2 Determine the migration approach (biggest task: 64)

Q: Move [servers / database / files / network] with minimal downtime / LEAST overhead / a deadline.
A: Right tool for what's moving + continuous replication before cutover, over existing connectivity.

Need	Answer
Database, minimal downtime	**DMS full load + CDC** (+ **SCT** for new engine)
Oracle + stored procedures → managed	**SCT + DMS → Aurora PostgreSQL**
MongoDB → managed	**DMS → DocumentDB**
VMs, short cutover	**MGN** (exam: SMS) + **DataSync** NFS → EFS
Files to FSx/EFS, incremental, verified	**DataSync**
On-prem keeps SMB/NFS, data in S3	**S3 File Gateway**

Need	Answer
iSCSI + low latency	**Volume Gateway (cached)**
SFTP with static IPs	**Transfer Family + Elastic IPs** (VPC endpoint)
Many VPCs + on-premises	**DX gateway + transit VIF + TGW**
Hybrid DNS	**Resolver endpoints + rules**
Landing zone with guardrails	**Control Tower** (+ org CloudTrail)
SMTP app → AWS	**SES SMTP credentials + STARTTLS**

Checks: right tool for this data? · CDC or one-time? · is the link already there?

4.3 New architecture for existing workloads

Q: Re-host the moved app to be highly available / scalable / low overhead without major changes.
A: A **managed target that keeps the app's protocol and engine**, multi-AZ.

Need	Answer
Containers, no servers, NFS share	**ECS on Fargate + EFS**
SQL Server app, cut licences	**Aurora PostgreSQL + Babelfish**
JAR / PHP app, little change	**Elastic Beanstalk** (+ RDS / Aurora replicas)
Same ECS tools on factory servers	**ECS Anywhere**
File system synced with S3	**FSx for Lustre** data repository

Need	Answer
Multiprotocol for EKS	**FSx for NetApp ONTAP**
Sessions highly available	**ElastiCache Redis (replicas)** / DynamoDB
Microsecond reads + durable	**MemoryDB Multi-AZ**
Partner allow-lists your IP	**BYOIP → EIPs on NAT** / **NLB static IPs → ALB**
DR back to on-prem later	**Elastic Disaster Recovery** (failback)

Checks: same engine and protocol? · multi-AZ? · fixed IP or licence limits?

4.4 Modernization

Q: Modernize the moved app: serverless / containers / Kubernetes / decoupled with LEAST overhead.
A: Move **step by step** to the **managed version of the same pattern**; containers for long jobs, serverless for short event-driven work.

Need	Answer
Lambda hits 15 min	**Container image → ECS on Fargate**
Existing Kubernetes manifests	**EKS managed nodes** (air-gapped: **EKS Anywhere**)
Must control OS / host / custom agent	**ECS or EKS on EC2** (not Fargate)
Web + API + DB to serverless	**S3 + Lambda/API Gateway + Aurora** (DMS)
Failed orders kept	**SQS + DLQ**
SFTP + transform + queue	**Transfer Family → S3 → Glue → SQS**

Need	Answer
RabbitMQ / AMQP, no change	**Amazon MQ**
SaaS data → S3, no code	**AppFlow** (+ PrivateLink)
Fast releases + rollback	**Beanstalk swap URLs** / blue/green
Keep agents updated on 10,000 instances	**State Manager association**
Short timeline, tightly coupled	**Lift and shift first, modernize later**

Checks: runtime over 15 min? · existing Kubernetes or OS needs? · step by step, not a rewrite?